

Colombo Eats

A Restaurant Review Application for Android

Module: COMP50072

Assignment 2

Student Name: [PLACEHOLDER]

Student ID: CB016808

Date: [PLACEHOLDER]

Table of Contents

1. Introduction	1
2. Development Approach and Architecture	1
3. Application Walkthrough	3
4. Interface Design and Accessibility.....	13
5. Responsive Layout	15
6. Firebase Realtime Database.....	16
7. Location Services.....	18
8. Testing.....	21
9. Problems Encountered and Decisions Made	21
10. Limitations and Future Work	22
11. References	22
Appendix A: Manual Test Plan	23
Appendix B: Firebase Realtime Database Security Rules.....	27

(Right-click this area and choose Update Field, or press F9, to build the table of contents.)

1. Introduction

Colombo Eats is a restaurant review app I built for Android with Flutter. It is meant for people in Colombo deciding where to eat, and for the people who run those restaurants. There is no single dominant local review app for the city; most existing options are general map or delivery apps with reviews added as an afterthought. This app puts reviewing at the centre instead.

The code recognises three kinds of user. A signed-in person can browse every restaurant and post their own review. A person who owns a restaurant can add it themselves, with a description, a photo and a location, and manage it afterwards. A signed-out visitor can do neither: `main.dart` gates the whole app behind Firebase authentication, so there is no way to browse or review without an account first.

2. Development Approach and Architecture

The app is built with Flutter 3.47.0 and Dart 3.13.0, targeting Android. I chose Flutter because it compiles one Dart codebase to a native Android binary (REF-01), which meant I did not need a second language or a separate build system for the coursework. The obvious cost is that it will not run on iOS without platform work I have not done.

`lib/` is organised by feature rather than by type. `main.dart` starts Firebase and decides between the login screen and the main app. `theme/` holds the single seed colour the whole palette comes from. `models/` has Restaurant and Review, the two record types stored in the database. `services/` wraps every Firebase and GPS call so screens never talk to Firebase directly. `utils/` has one function, `ratingsByRestaurant`, which turns a list of reviews into an average. `shared_widgets/` holds the card and image widgets used on more than one screen. `screens/` is split into a subfolder per feature: `auth`, `home`, `restaurants`, `my_reviews`, `my_restaurants` and `profile` (Figure 1).

State management is `setState` throughout, and navigation is `Navigator.push` and `Navigator.pop`, with no Provider, Riverpod, Bloc or `go_router` anywhere in the project. This was a deliberate limit. I wanted to be able to explain every part of the app from memory in a viva, and a state management package would have meant defending someone else's abstraction rather than my own decisions.

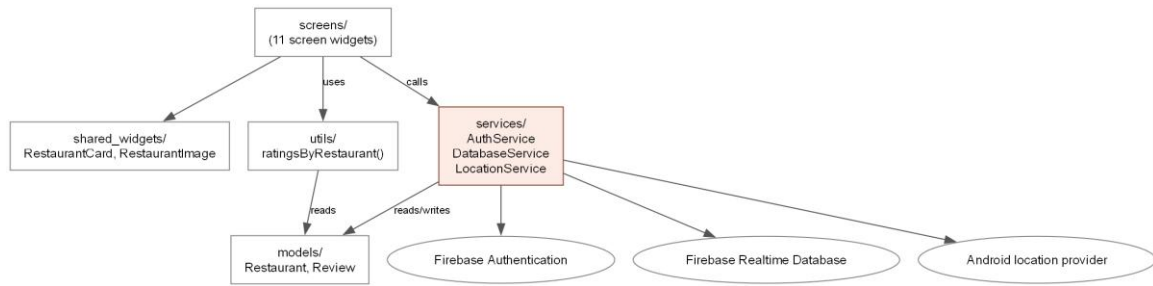


Figure 1: lib/ folder structure and how the layers depend on each other.

3. Application Walkthrough

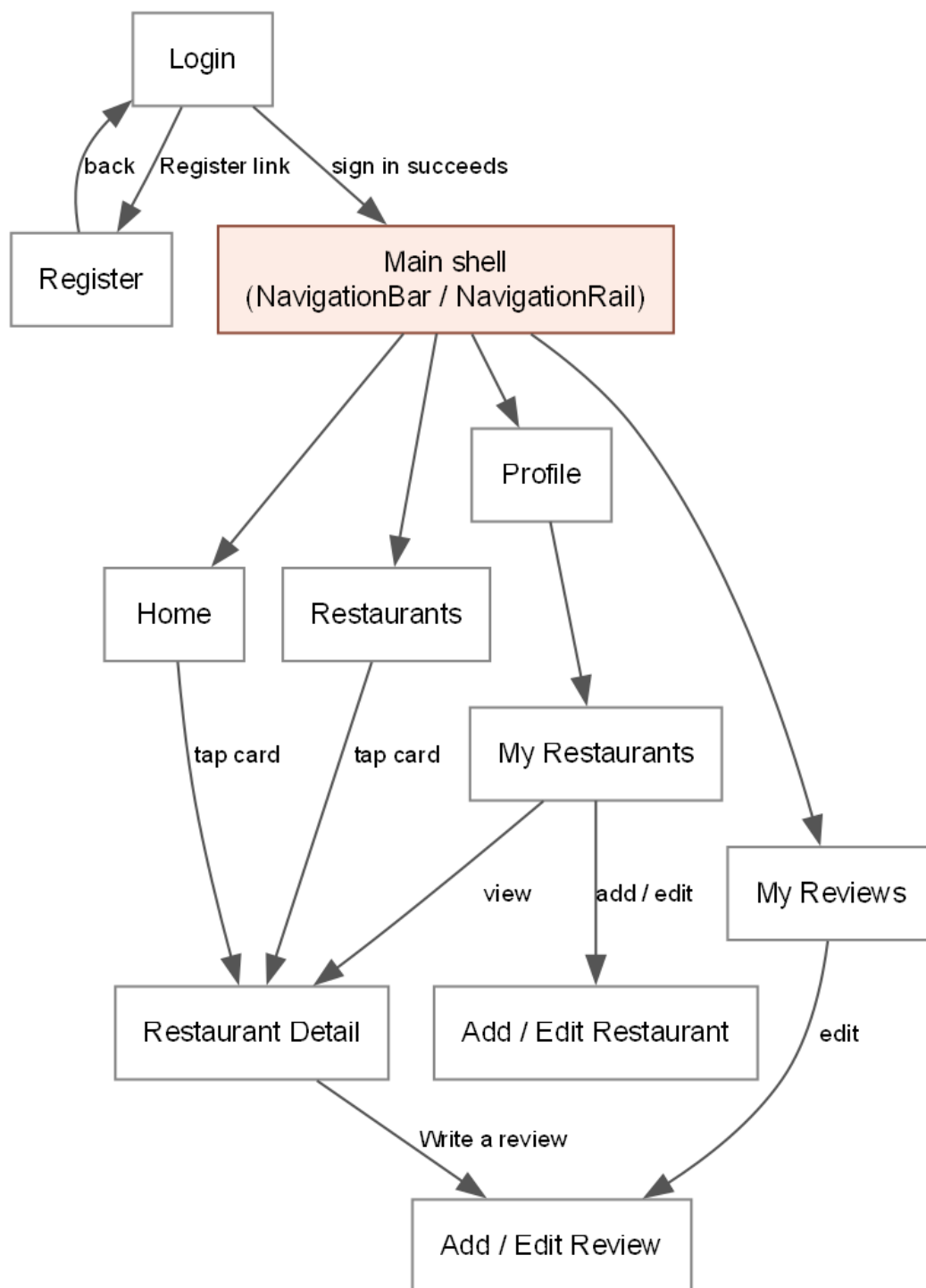


Figure 2: navigation map, showing every screen and how a user reaches it.

Login is the first screen a signed-out user sees, or the app opens straight past it if a session was already saved on the device. It checks the email format and asks for a password of at least six characters before it will call Firebase, and shows a plain sentence rather than a raw error code when sign-in fails.

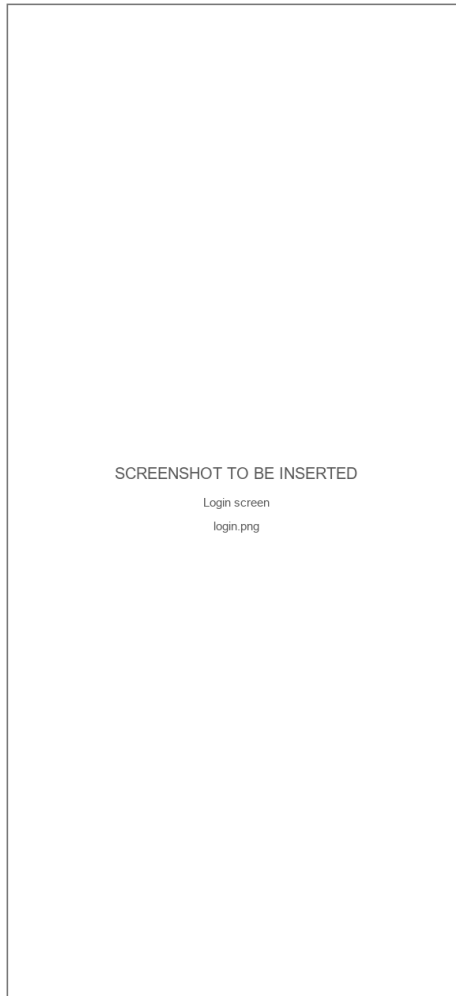


Figure 3: Login screen. [SCREENSHOT TO BE INSERTED: login.png]

Register collects a name, email, password and a confirmation field, creates the account, and writes the entered name onto it as a display name so later reviews can show who wrote them.

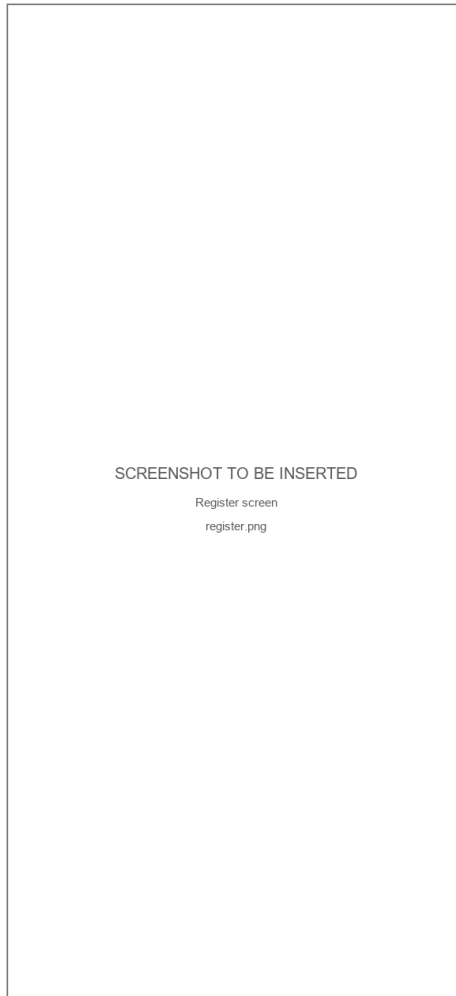


Figure 4: Register screen. [SCREENSHOT TO BE INSERTED: register.png]

Home lists the four highest-rated restaurants, ranked by the live average from their reviews rather than a stored number. A restaurant with no reviews has nothing to rank by and sorts last.

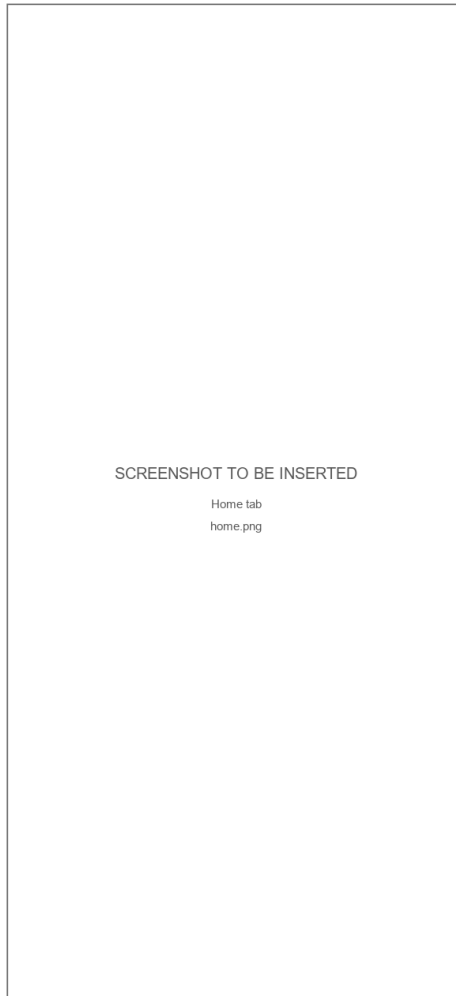
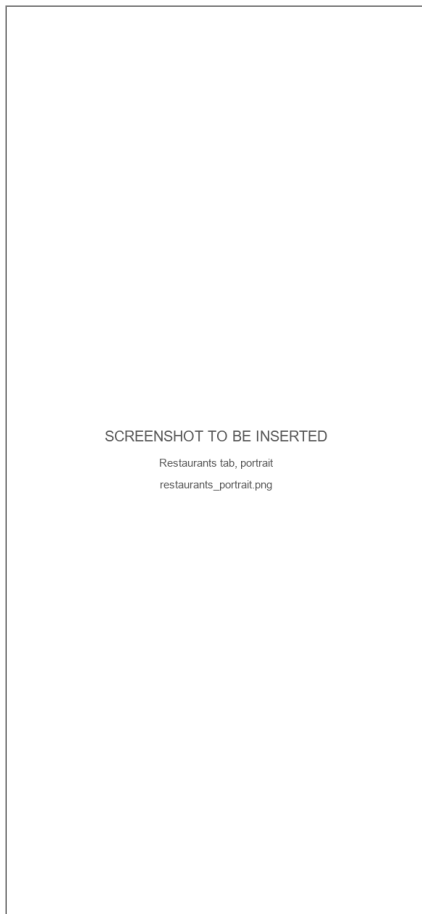


Figure 5: Home tab. [SCREENSHOT TO BE INSERTED: home.png]

Restaurants is the full list, with a slider to filter by minimum rating and a button that reads the device's GPS position and sorts by distance from it. The layout itself changes with orientation, covered in section 5.



*Figures 6 and 7: Restaurants tab, portrait and landscape. [SCREENSHOT TO BE INSERTED:
restaurants_portrait.png, restaurants_landscape.png]*

Restaurant Detail shows the description, price range, popular dishes and the underlying reviews, none of which appear on the card. If the signed-in account owns the restaurant, the review button is replaced with a banner instead, explained fully in section 6.

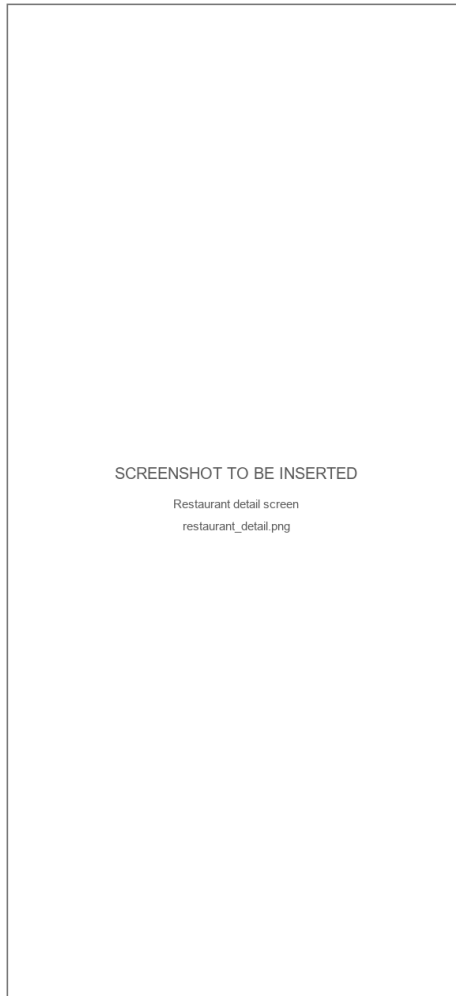


Figure 8: Restaurant Detail screen. [SCREENSHOT TO BE INSERTED: restaurant_detail.png]

Write a Review is one form used for creating and editing, with four field types: a slider for the rating, a dropdown for the meal, a date picker with no future dates allowed, and a text box for the comment capped at 400 characters.

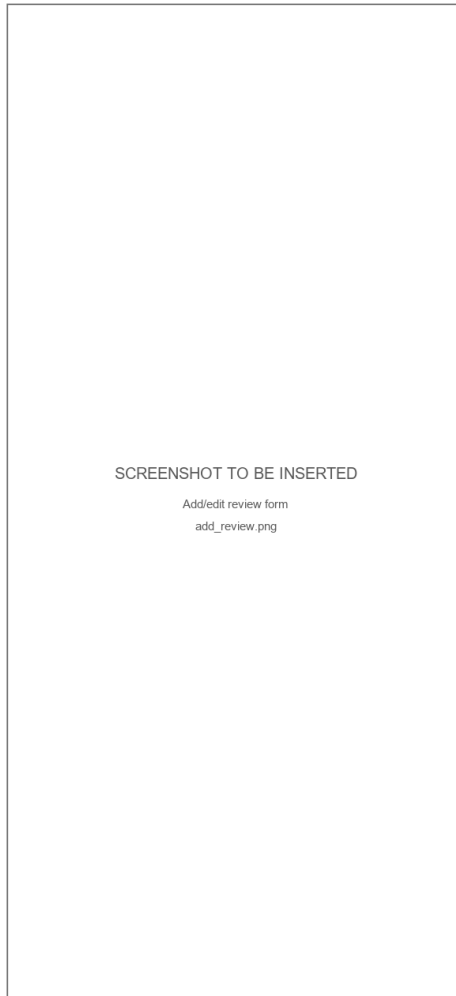


Figure 9: Add/edit review form. [SCREENSHOT TO BE INSERTED: add_review.png]

My Reviews lists every review the signed-in account has written, with edit and delete on each behind a confirmation dialog.

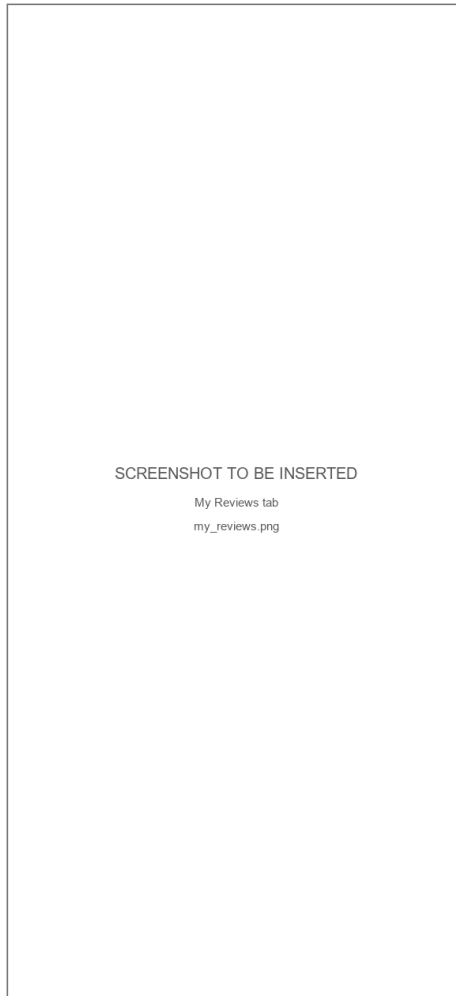


Figure 10: My Reviews tab. [SCREENSHOT TO BE INSERTED: my_reviews.png]

My Restaurants, reached from Profile, lists the restaurants the signed-in account owns and is where a user adds a new one.

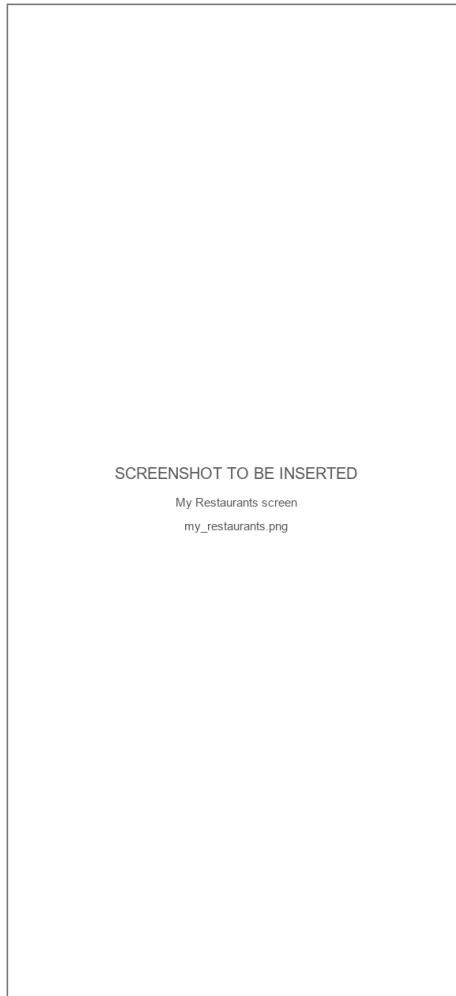


Figure 11: My Restaurants screen. [SCREENSHOT TO BE INSERTED: my_restaurants.png]

Add a Restaurant collects the name, cuisine, area, price, a photo link, dishes and a description, and has a button that fills latitude and longitude from a GPS fix rather than asking for typed coordinates. There is no rating field: an owner cannot score their own restaurant.

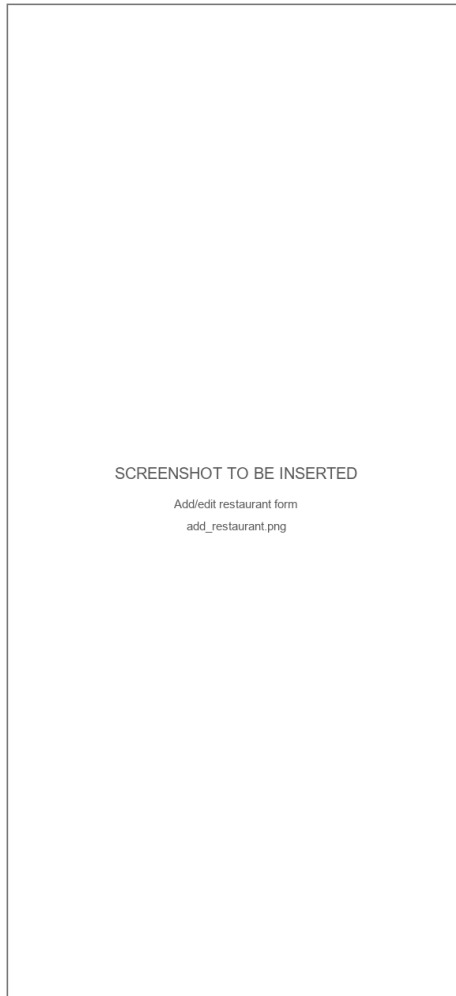


Figure 12: Add/edit restaurant form. [SCREENSHOT TO BE INSERTED: add_restaurant.png]

Profile shows the account's name and email, a count of reviews written, the average rating given, a link into My Restaurants, and sign out.

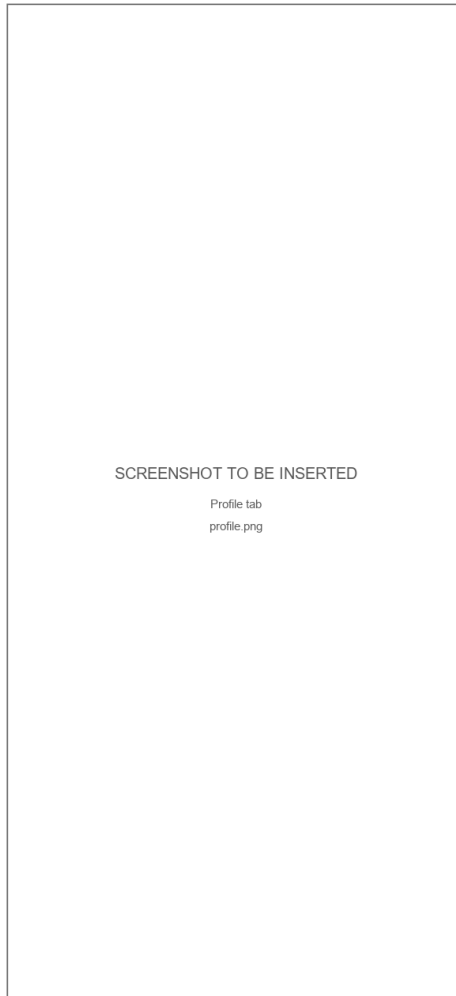


Figure 13: Profile tab. [SCREENSHOT TO BE INSERTED: profile.png]

4. Interface Design and Accessibility

The whole app uses Material 3 and generates both colour schemes from one seed colour, a deep orange, `0xFFBF360C`, with `ColorScheme.fromSeed` (REF-02). `themeMode` is set to `ThemeMode.system`, so the app follows whatever light or dark setting is already on the phone.

I did not want to assume the seed colour produced readable text, so I wrote a script using the same `ColorScheme.fromSeed` call the app uses and measured the WCAG contrast ratio (REF-03) for eight foreground and background pairs, in both light and dark mode.

Table 1: measured WCAG contrast ratios, `ColorScheme.fromSeed(0xFFBF360C)`

Pair	Light mode	Dark mode	Threshold
onSurface / surface	16.36 : 1	14.42 : 1	4.5 : 1
onSurfaceVariant / surface	8.92 : 1	10.93 : 1	4.5 : 1
onPrimary / primary	6.49 : 1	7.74 : 1	4.5 : 1

Pair	Light mode	Dark mode	Threshold
onPrimaryContainer / primaryContainer	7.25 : 1	7.25 : 1	4.5 : 1
onSecondaryContainer / secondaryContainer	7.23 : 1	7.23 : 1	4.5 : 1
onError / error	6.46 : 1	7.72 : 1	4.5 : 1
onErrorContainer / errorContainer	7.24 : 1	7.24 : 1	4.5 : 1
outline / surface	4.28 : 1	5.84 : 1	3.0 : 1

Every pair cleared the 4.5:1 threshold recommended for normal text, and outline against surface, checked against the lower 3:1 threshold used for borders, cleared that too. The lowest of the sixteen ratios measured was outline against surface in light mode, at 4.28 to 1. I should be honest that this checks the theme's own numbers rather than how the screen looks in daylight, which I have not tested on a physical device.

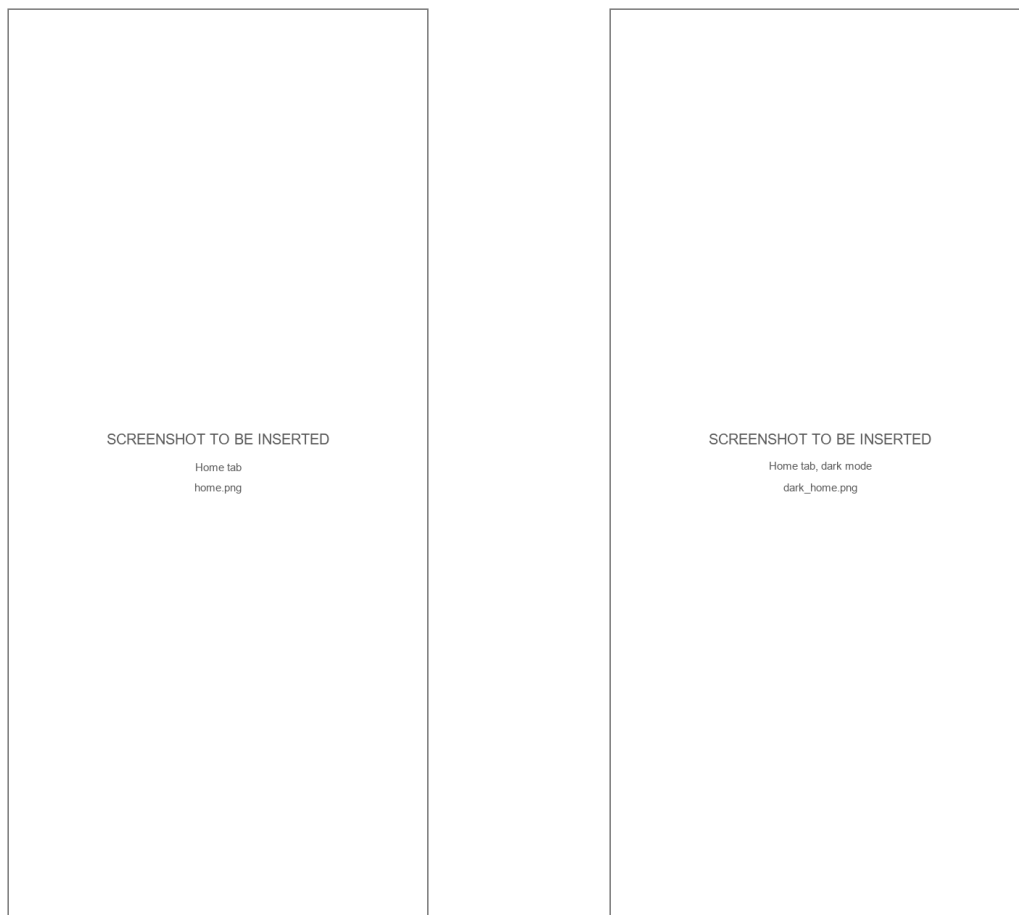


Figure 14: Home tab, light and dark mode. [SCREENSHOT TO BE INSERTED: home.png, dark_home.png]

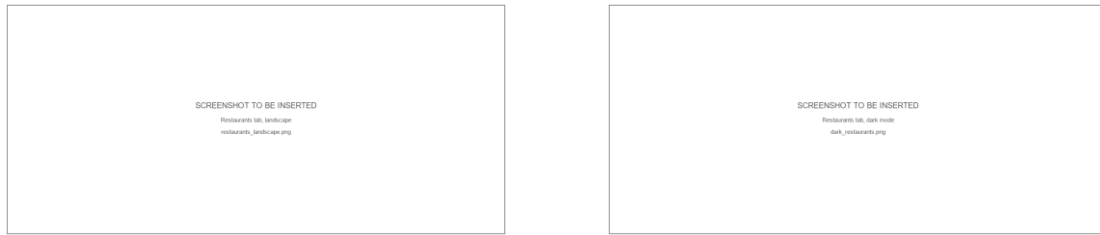


Figure 15: Restaurants tab, light and dark mode. [SCREENSHOT TO BE INSERTED: restaurants_landscape.png, dark_restaurants.png]

5. Responsive Layout

Two screens use OrientationBuilder to change layout on rotation. On Restaurants, portrait shows a single column of wide cards in a ListView.builder; landscape switches to a GridView.builder using a different, more compact card widget entirely, RestaurantGridCard rather than RestaurantCard. On Restaurant Detail, portrait stacks the photo above scrolling text in a Column; landscape puts the photo in a fixed panel on the left with the text scrolling on its own in a separate SingleChildScrollView on the right, which changes the structure rather than just the size.

Screen size is handled separately, with MediaQuery.of(context).size.shortestSide, and I want to describe accurately what it does: it changes the column count on the Restaurants grid, between one, two or three depending on width and orientation, and swaps the bottom NavigationBar for a side NavigationRail once shortestSide passes 600 logical pixels. It does not give the tablet a separate page design beyond that.



Figure 16: phone portrait, phone landscape, tablet portrait and tablet landscape compared. [SCREENSHOT TO BE INSERTED: restaurants_portrait.png, restaurants_landscape.png, tablet_portrait.png, tablet_landscape.png]

6. Firebase Realtime Database

All app data sits in one Firebase Realtime Database under two top-level nodes, restaurants and reviews. A restaurant record holds its name, cuisine, area, price range, an image URL, a description, an optional list of popular dishes, latitude, longitude, and an ownerId, empty for the eight restaurants I imported to seed the database and set to a Firebase Auth uid for anything a user adds.

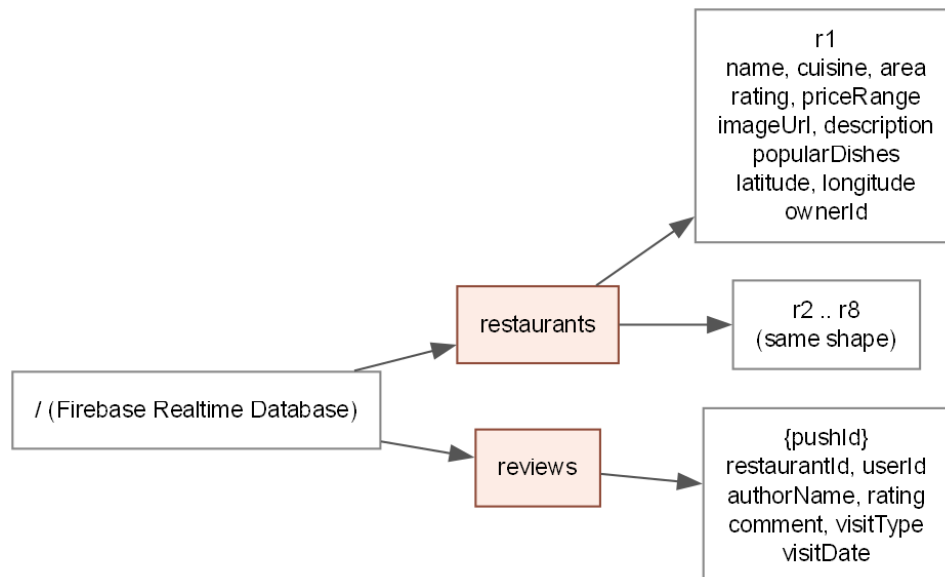


Figure 17: shape of the data under restaurants/ and reviews/.

I used the Realtime Database rather than Firestore because it was what the coursework covered, and I read every list through an `.onValue` stream (REF-05) rather than a one-off `.get()`, so a change made from the console or another device appears immediately with no manual refresh anywhere in the app.

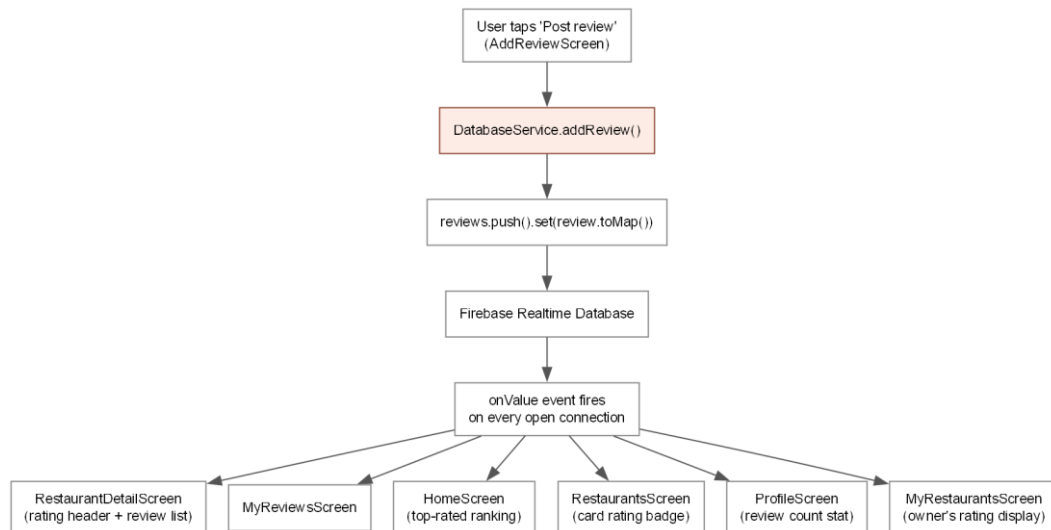


Figure 18: one write, from button press to every screen that updates because of it.

Table 2: the four CRUD operations, both entities, where each happens

Operation	Entity	Screen / widget	DatabaseService method
Create	Review	AddReviewScreen, Post review button	addReview()
Read	Review	RestaurantDetailScreen, review list	reviewsStream()
Update	Review	MyReviewsScreen, edit icon	updateReview()
Delete	Review	MyReviewsScreen, delete icon	deleteReview()
Create	Restaurant	AddRestaurantScreen, Add restaurant button	addRestaurant()
Read	Restaurant	RestaurantsScreen / HomeScreen list	restaurantsStream()
Update	Restaurant	MyRestaurantsScreen, edit icon	updateRestaurant()
Delete	Restaurant	MyRestaurantsScreen, delete icon	deleteRestaurant()

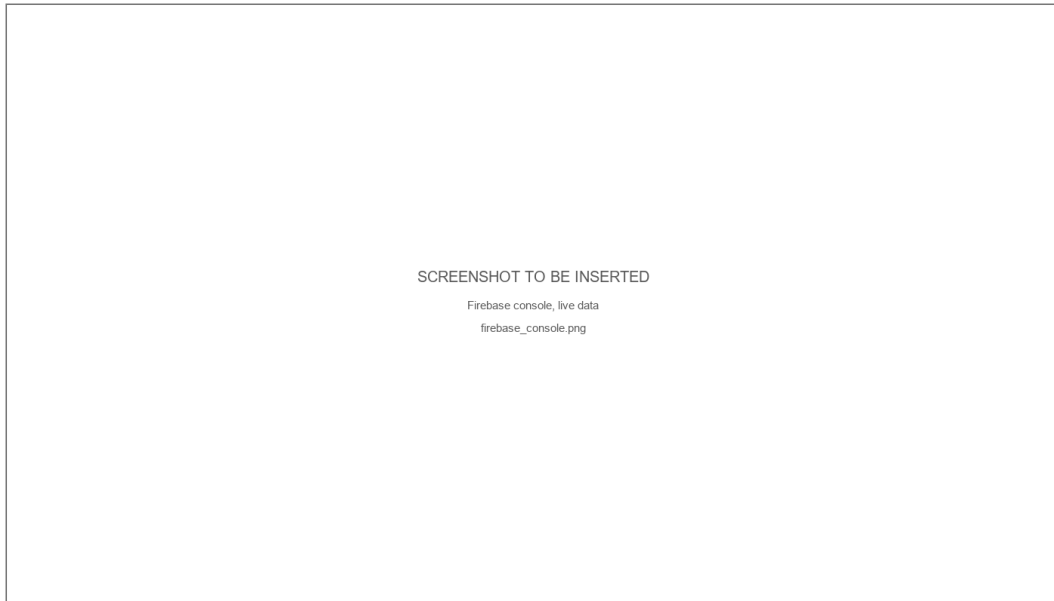


Figure 19: Firebase console showing the live restaurants and reviews nodes. [SCREENSHOT TO BE INSERTED: firebase_console.png]

The deployed rules (REF-04, Appendix B) refuse every read and write by default, then open both nodes to read for a signed-in user only. A restaurant can only be written by the uid in its own ownerId field, on creation and every update after, which stops one user editing or deleting somebody else's listing. A review can only be created if the writing uid matches its own userId and is not the ownerId of the restaurant being reviewed, which is what actually stops an owner reviewing their own place, not just the app hiding a button. The rating field has its own rule: it must be exactly zero on creation and cannot change on any update, which stops an owner setting or later raising their own score by writing to Firebase directly.

7. Location Services

GPS is used for two things. On Restaurants, tapping the compass icon reads the current position, sorts every restaurant by distance and shows a badge on each card. On Add Restaurant, a button fills latitude and longitude from the same read, so an owner can tag a restaurant by standing in it.

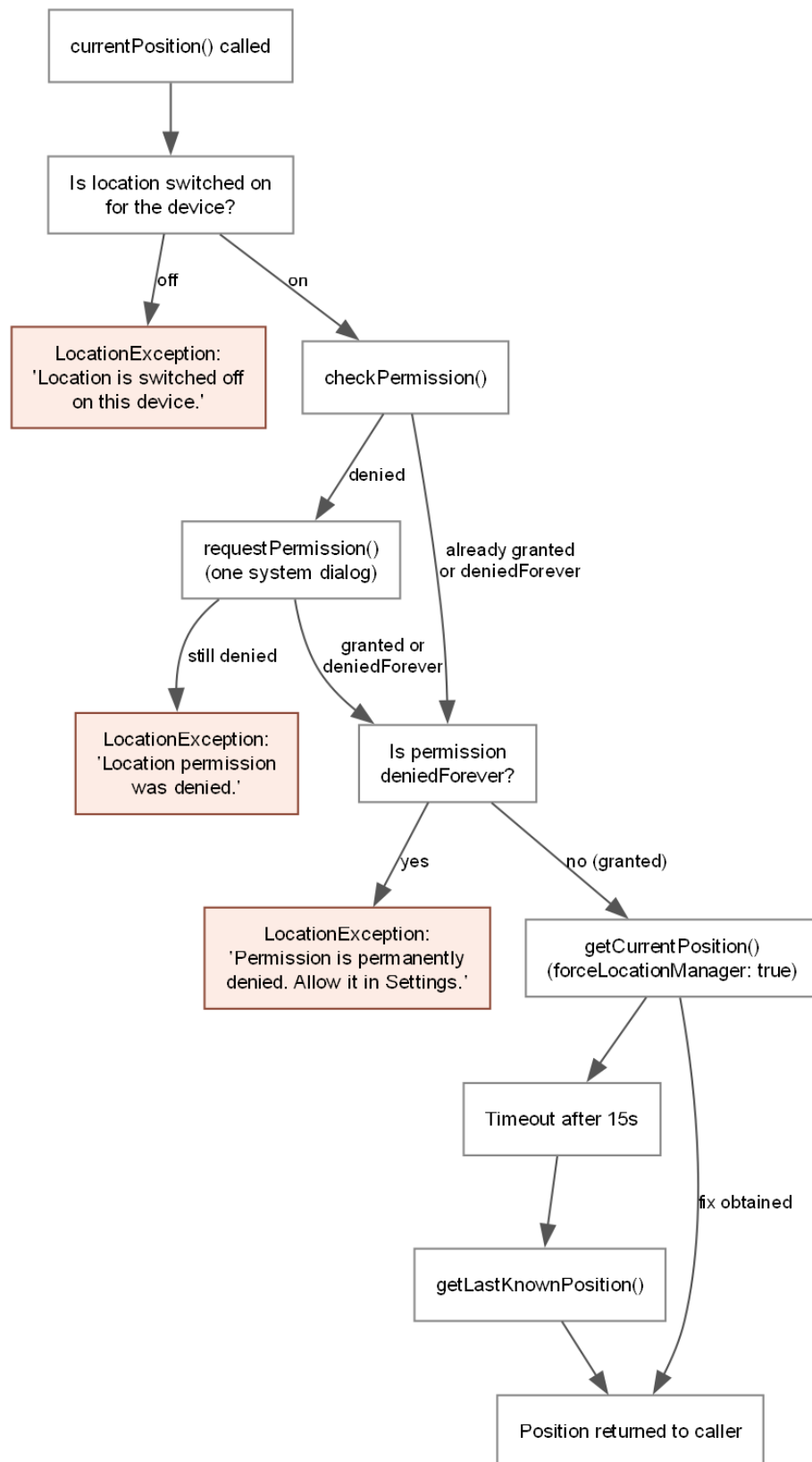
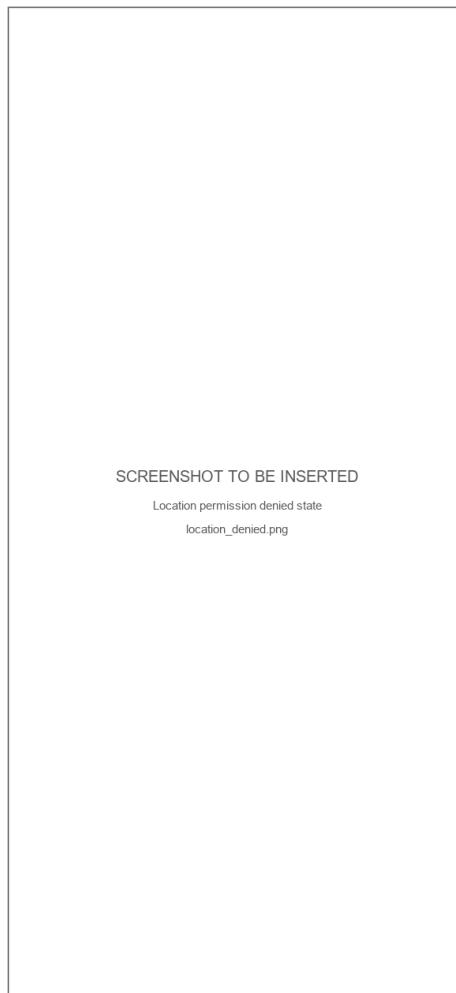


Figure 20: the three permission states LocationService checks, and the message shown for each.

`LocationService.currentPosition()` checks three states before reading anything: whether location is switched on, whether permission has been granted, asking once if not, and whether permission has been permanently denied, which needs a message pointing the user to Android's own Settings since the system dialog will not appear again (REF-07). Each state gets its own message rather than one generic failure.



*Figure 21: the message shown when location permission is denied. [SCREENSHOT TO BE INSERTED:
location_denied.png]*

While building this I found the first tap after opening the app often reported a position kilometres from the real one, with a second tap then correct. Android's fused location provider was returning a cached fix it judged recent enough (REF-06). Setting `forceLocationManager: true` fixed it by forcing the read through Android's own `LocationManager` instead, at some cost in speed. One limitation stands: there is no way to type coordinates by hand, so a user who refuses location permission cannot add a restaurant, though they can still browse and review.

8. Testing

test/widget_test.dart has 14 unit tests, all synchronous and none touching Firebase. They check that Restaurant.fromMap and Review.fromMap parse a complete record, fall back safely when a field is missing, and handle a rating Firebase returns as a whole number. A second group checks ownership on Restaurant, including the case where a signed-out uid of an empty string must not be treated as owning a restaurant whose ownerId is also empty. A third group checks ratingsByRestaurant directly: an empty list gives an empty map, a restaurant with no reviews has no entry rather than a zero, and an average of 4.0, 4.0 and 5.0 rounds to 4.3.

Table 3: test coverage summary

Area	Automated (unit tests)	Manual (test plan rows)
Model parsing (Restaurant, Review)	5	0
Ownership logic	4	4 (section 5)
Live rating calculation	5	1 (O4)
Authentication	0	11 (section 1)
Navigation	0	5 (section 2)
Review CRUD via UI	0	12 (section 3)
Restaurant CRUD via UI	0	6 (section 4)
Form validation	0	8 (section 6)
GPS sensor	0	6 (section 7)
Layout, theme, animation	0	12 (section 8)
Total	14	64

This is model and logic testing only. There is no widget testing and no integration testing anywhere in the project; nothing pumps a screen or taps a button inside a test. That was a scope decision given the time available. The manual test plan (Appendix A) is what covers screen behaviour, navigation, the Firebase rules and the sensor, run by hand on an emulator rather than automated.

9. Problems Encountered and Decisions Made

A few things did not work the first time.

Deleting a restaurant originally cascaded to delete every review about it, which I built before thinking it through. If an owner could remove reviews by deleting and recreating their restaurant, nothing stopped them wiping a bad one that way. I removed the cascade: DatabaseService.deleteRestaurant now only removes the restaurant, and reviews left pointing at one that no longer exists are filtered out in MyReviewsScreen instead.

A restaurant's rating was, for most of the project, a number stored on the record and set once at creation. It never changed when a new review came in, which I did not notice until I asked myself directly whether the number on screen would move after posting a review. It would not have. I replaced it with `ratingsByRestaurant`, computed live from the reviews stream on every screen that shows a rating.

The rules stopping an owner reviewing their own restaurant and setting their own rating sat in `firebase/database_rules.json` for a while before I deployed them with `firebase deploy --only database`. Until that command ran, the live database was still enforcing an older, looser rule set, so the add-restaurant feature accepted input in the UI while every write was silently rejected by Firebase underneath it.

The GPS bug from section 7 belongs here too. `forceLocationManager` was not something I knew to reach for; I found it after noticing the wrong-then-right pattern on two consecutive taps and searching for why Android's fused provider would behave that way.

10. Limitations and Future Work

Distance shown to the user is a straight line between two GPS points, not a walking or driving distance, so it understates how far a restaurant is if the road network is not direct. Nothing stops one account posting more than one review for the same restaurant. Adding a restaurant depends entirely on a GPS fix with no manual entry, so it cannot be done without granting location permission. Given more time I would add typed address entry as a fallback and some limit on repeat reviews, but neither exists now and I am not listing them as though they were planned features.

11. References

The reference list for this report will be inserted here once the sources named in docs/report/REFERENCES_NEEDED.md have been found and formatted in Harvard style.

Appendix A: Manual Test Plan

Carried out on an Android emulator (Pixel 6, API 34), Flutter 3.47.0, Dart 3.13.0, against the Firebase Realtime Database in asia-southeast1, debug build.

A.1 Authentication

ID	Steps	Expected	Actual	Pass/Fail
A1	Register → fill all fields → Create account	Account created, app opens on Home tab		
A2	Enter nethusha.com → submit	Field error "Enter a valid email address"; nothing sent to Firebase		
A3	Enter 123 → submit	Field error "Password must be at least 6 characters"		
A4	Enter different confirm password	Field error "Passwords do not match"		
A5	Use an email already registered	Red bar: "An account already exists with that email"		
A6	Enter valid credentials → Sign in	Opens on Home tab		
A7	Enter valid email, wrong password	Red bar: "Email or password is incorrect"		
A8	Tap Sign in with both fields blank	Both fields show validation errors		
A9	Sign in → close app → reopen	Opens signed in, no login screen		
A10	Profile → Sign out → confirm	Returns to login screen		
A11	Profile → Sign out → Cancel	Stays signed in		

A.2 Navigation

ID	Steps	Expected	Actual	Pass/Fail
N1	Tap each destination in the bottom bar	Correct screen shown, icon fills in		
N2	Restaurants → tap any card	Detail screen opens with that restaurant		
N3	Detail → back arrow	Returns to the list, scroll position kept		
N4	Login → Register	Register screen opens		
N5	Register → back arrow	Returns to login		

A.3 Firebase CRUD — reviews

ID	Steps	Expected	Actual	Pass/Fail
C1	Open Restaurants tab	8 restaurants load from Firebase		
C2	Detail → Write a review → fill → Post	Snackbar "Review posted"; appears under Reviews immediately		
C3	My Reviews tab	The new review is listed		

ID	Steps	Expected	Actual	Pass/Fail
C4	My Reviews → edit icon → change rating → Save changes	Snackbar “Review updated”; new values shown		
C5	My Reviews → delete icon → Delete	Snackbar “Review deleted”; row disappears		
C6	Delete icon → Cancel	Review is not deleted		
C7	Add a review in the Firebase console while the app is open	Appears in the app without any refresh		
C8	Sign in as a second user, view a restaurant	Other users’ reviews visible but no edit/delete buttons on My Reviews		
C9	Open Restaurants on a cold start	Spinner shown before the list appears		
C10	Turn off emulator network → open Restaurants	“Could not load restaurants” message, no crash		
C11	Set the rating filter to 5.0	“No restaurants match” message		
C12	Firebase console → Rules → Simulator → write to /reviews unauthenticated	Simulation denied		

A.4 Firebase CRUD — restaurants

ID	Steps	Expected	Actual	Pass/Fail
R1	Profile → My Restaurants → Add restaurant → fill form → Use my current location → Add restaurant	Snackbar “Restaurant added”; listing appears in the Restaurants tab with a “New” chip		
R2	My Restaurants → edit icon → change the name → Save changes	Snackbar “Restaurant updated”; new name shown everywhere the restaurant appears		
R3	My Restaurants → delete icon → Delete	Snackbar “Restaurant deleted”; listing disappears from Restaurants and Home; any reviews written about it are not deleted and simply stop being displayed		
R4	Add restaurant → leave the photo link blank → Add restaurant	Field error “Paste a photo link for your restaurant”		
R5	Add restaurant → type a description under 20 characters	Field error “Please write at least 20 characters”		
R6	Add restaurant → do not tap “Use my current location” → Add restaurant	Inline error: “Set the restaurant location before saving.” — save is blocked		

A.5 Ownership rules

ID	Steps	Expected	Actual	Pass/Fail
O1	Open a restaurant you added yourself	No “Write a review” floating button; a banner reads “This is your restaurant.”		

ID	Steps	Expected	Actual	Pass/Fail
		Owners cannot review their own listings.”		
O2	While signed in as the owner, attempt a direct write to /reviews with restaurantId set to your own restaurant and userId set to your own uid	Write is rejected — PERMISSION_DENIED		
O3	While signed in as the owner, attempt a direct write to /restaurants/{yourId}/rating setting any value other than the current one	Write is rejected — PERMISSION_DENIED, both for a non-zero value on create and for any change on update		
O4	Note a restaurant’s current rating (or “New”) → post a review for it as a different account → return to the Restaurants list and detail page	The average and review count update immediately, with no app restart		

A.6 Form validation

ID	Steps	Expected	Actual	Pass/Fail
F1	Post review with blank comment	“Write a few words about your visit”		
F2	Enter Good as the comment	“Please write at least 10 characters”		
F3	Drag the rating slider from 1.0 to 5.0	Value updates live next to the slider		
F4	Open the meal dropdown, choose Lunch	Selection shown in the field		
F5	Tap the date field, choose a past date	Date shown as e.g. “12 Aug 2026”		
F6	Open the date picker	Dates after today cannot be selected		
F7	Type past 400 characters in the review comment	Counter stops at 400		
F8	My Reviews → edit	Rating, meal and comment already filled in		

A.7 Hardware sensor — GPS

ID	Steps	Expected	Actual	Pass/Fail
G1	First tap of the location icon	Android permission dialog appears		
G2	Allow	Distance badges appear; list sorts nearest first		
G3	Deny	Red bar explaining distances will not show; no crash		
G4	Emulator settings → Location off → tap the icon	Message “Location is switched off on this device”		
G5	Set emulator to 6.9271, 79.8612	Colombo 01–07 restaurants all within a few km		

ID	Steps	Expected	Actual	Pass/Fail
G6	Tap the close icon on the location bar	Bar closes, distances stay		

A.8 Layout, theme and animation

ID	Steps	Expected	Actual	Pass/Fail
L1	Device Settings → Display → Dark theme	App recolours with no restart		
L2	Turn dark theme back off	App returns to light palette		
L3	Restaurants tab, upright	Single-column list of wide cards		
L4	Rotate	Two-column grid of compact cards		
L5	Run on a Pixel Tablet AVD, portrait	NavigationRail on the left instead of the bottom bar; two-column grid		
L6	Rotate the tablet	Three-column grid; NavigationRail remains		
L7	Open a restaurant upright	Photo on top, details underneath		
L8	Rotate	Photo on the left, details scroll on the right		
L9	Tap a card	Photo expands from the card into the detail screen		
L10	Tap the filter icon	Panel slides open rather than snapping		
L11	Break an imageUrl in Firebase	Grey box with a fork icon, no red error box		
L12	Set a restaurant name to 60 characters	Name truncates with an ellipsis, layout does not break		

Total: 64 manual cases across 8 areas. Actual result, Pass/Fail and Date columns are completed by hand while testing on device and are not reproduced here in full; the source table with those columns lives in docs/TEST_PLAN.md.

Appendix B: Firebase Realtime Database Security Rules

The rules below are deployed to the live database and enforced by Firebase on every read and write, independent of the app's own UI.

```
{
  "rules": {
    ".read": false,
    ".write": false,

    "restaurants": {
      ".read": "auth != null",

      "$restaurantId": {
        ".write": "auth != null && (!data.exists() ? newData.child('ownerId').val() === auth.uid
: data.child('ownerId').val() === auth.uid)",

        ".validate": "newData.hasChildren(['ownerId', 'name', 'cuisine', 'area', 'description',
'imageUrl', 'latitude', 'longitude'])",

        "ownerId": {
          ".validate": "newData.val() === auth.uid"
        },
        "name": {
          ".validate": "newData.isString() && newData.val().length >= 2 && newData.val().length
<= 80"
        },
        "description": {
          ".validate": "newData.isString() && newData.val().length >= 20 && newData.val().length
<= 400"
        },
        "rating": {
          ".validate": "newData.isNumber() && newData.val() >= 0 && newData.val() <= 5 &&
(!data.exists() ? newData.val() === 0 : newData.val() === data.val())"
        },
        "latitude": {
          ".validate": "newData.isNumber() && newData.val() >= -90 && newData.val() <= 90"
        },
        "longitude": {
          ".validate": "newData.isNumber() && newData.val() >= -180 && newData.val() <= 180"
        }
      }
    },

    "reviews": {
      ".read": "auth != null",

      "$reviewId": {
        ".write": "auth != null && (!data.exists() ? (newData.child('userId').val() === auth.uid
&& root.child('restaurants').child(newData.child('restaurantId').val()).child('ownerId').val()
!== auth.uid) : data.child('userId').val() === auth.uid)",

        ".validate": "newData.hasChildren(['restaurantId', 'userId', 'authorName', 'rating',
'comment', 'visitType', 'visitDate'])",

        "rating": {
          ".validate": "newData.isNumber() && newData.val() >= 1 && newData.val() <= 5"
        },
        "comment": {
          ".validate": "newData.isString() && newData.val().length >= 10 && newData.val().length
<= 400"
        },
        "userId": {
          ".validate": "newData.val() === auth.uid"
        }
      }
    }
  }
}
```

}
}